Windowed Edge Aggregation and Batched Serverless Ingestion for a Two-Berth Smart Port Container Terminal Monitor

Uday Kiran Reddy Dodda
Student ID: X25166484
MSc in Cloud Computing
Fog and Edge Computing (H9FECC)
National College of Ireland
x25166484@student.ncirl.ie

***Abstract*—A container terminal runs cranes, container stacks, and refrigerated cargo against tight schedules, yet the condition of a berth is often known only from operator reports rather than live measurement, so a crane overload, a wind gust that should halt lifting, or a warming reefer can develop unseen between checks. This paper presents a continuous two-berth terminal-monitoring pipeline. Five sensor types per berth feed a fog gateway that windows and aggregates readings and evaluates safety thresholds locally, forwarding only compact summaries to the cloud. A scalable serverless backend on Amazon SQS, AWS Lambda, and DynamoDB ingests those summaries and serves a live operations dashboard through Amazon API Gateway, with the static frontend hosted on Amazon S3. Each layer is justified against the alternatives that were weighed and set aside, and the system is deployed to a real AWS account rather than only a local emulator. An automated suite of 95 tests passes, and end-to-end verification against the live deployment confirmed every pipeline health check reporting healthy, fresh readings arriving within seconds, and both berths' metrics and a firing crane-overload alert rendering correctly on the deployed dashboard.**

***Keywords*—smart port, container terminal, fog computing, edge computing, Internet of Things, serverless computing, Amazon SQS, AWS Lambda, Amazon DynamoDB**

I. Introduction

A container terminal is a dense, safety-critical worksite. Quay cranes lift boxes weighing tens of tonnes over a berth, container stacks rise and fall as yard equipment works them, refrigerated containers hold perishable cargo within a narrow temperature band, and lifting has to pause when wind speed climbs past a safety limit. Operations research on container terminals has long treated berth productivity and equipment scheduling as the central levers of terminal throughput [1], but a scheduling decision is only as good as the terminal's awareness of its own current state. When that awareness comes from periodic operator reports rather than continuous measurement, a crane drifting toward an overload condition, a berth congesting past a workable occupancy, or a reefer warming toward a cold-chain breach can each escalate in the interval between one check and the next.

Instrumenting the terminal with always-on sensors closes that interval, and the wider move to instrument ports this way is well established: a survey of Internet of Things technologies for smart ports identifies dense sensing across cranes, yards, and cold-chain assets, together with the data handling it demands, as the defining characteristic of the smart port [2]. The remaining question for a distributed-systems design is where the readings are processed and how they travel from the quay to the people who act on them. Streaming every raw sample from every sensor to a distant region wastes bandwidth on readings that individually rarely matter and delays exactly the moment a threshold breach needs to be flagged. Fog computing was proposed to address exactly this, placing an intermediate compute tier between the sensors and the cloud, close enough to the equipment to summarise, filter, and evaluate rules before anything crosses the network [3], and it is one expression of the broader shift of computation back toward the network edge [4]. The cloud tier behind that fog layer provides the on-demand, elastically provisioned, pay-per-use model that reshaped how backends are built [5], so it absorbs bursts and idles at no cost without anyone pre-sizing a server.

This project builds that two-tier design for a scaled-down but structurally faithful terminal: two berths, designated berth-a and berth-b, each carrying the same five sensor types (crane load, container stack height, wind speed, berth occupancy, and reefer temperature) running as ten independent containers. A fog gateway ingests their readings, windows and aggregates each berth-and-sensor group on a fixed interval, evaluates threshold rules against the aggregate, and forwards only the finished window and any alerts onward. A cloud-side pipeline persists that window, and a dashboard folds the persisted history into a per-berth operational view alongside a cross-berth trend.

The work sets four objectives, each to be demonstrated running rather than only described. First, a sensor and fog layer that genuinely windows and alerts across five sensor types at independently configurable sampling and dispatch rates, not a rate fixed in code. Second, a backend that scales through managed, elastic AWS primitives instead of a single always-on server. Third, a dashboard that turns the stored windows

into an operational verdict a terminal supervisor could act on without reading raw values. Fourth, and distinct from the local emulator most development iterates against, deployment onto a real public-cloud account and verification directly against the live endpoint. The scope is deliberately bounded to these layers at a two-berth scale rather than a full multi-berth terminal. The remainder of this paper is organised as follows. Section II sets out the architecture and the reasoning behind each component choice, including the alternatives weighed and the security posture. Section III covers the implementation, the automated test suite, continuous integration, the real deployment together with the defects a pre-deployment audit corrected, and the evidence gathered from the running system. Section IV closes with a critical assessment of the design's trade-offs, its limitations, and where the work could go next.

II. System Architecture and Design

Fig. 1 shows the deployed system end to end: ten sensor containers and the fog gateway sharing one edge host, a message queue, two independently deployed Lambda functions, a DynamoDB table, and a browser-facing S3 bucket reached through API Gateway.

A. Sensor and Fog Layer

Each of the ten sensor containers represents one sensor type on one berth. A container advances a bounded random walk from its previous value on each sample, clamped to a physically plausible range for its metric, so consecutive readings stay close together like a real instrument trace rather than independent noise; this is what makes the windowed minimum, maximum, and average, and the dashboard trend line, resemble genuine terminal telemetry. Every container is driven by two independent, separately configurable intervals, one governing how often a new value is sampled and the other how often the accumulated batch is dispatched to the fog gateway, so a fast-changing quantity can be sampled frequently while still being sent in economical batches. At the deployment's default two-second sampling and ten-second dispatch cadence each dispatch carries roughly five readings as a small object naming the sensor type, the berth, the unit, and the timestamped values, well under a kilobyte on the wire. Within a container the two duties are structured as two self-rescheduling one-shot tasks sharing a single-thread scheduler, so the sample and dispatch actions are serialised onto one thread and the shared buffer needs no lock; on a dispatch failure the drained batch is simply prepended back onto the buffer for the next attempt, which is race-free precisely because no other thread can touch the buffer while that task runs.

The fog gateway is a plain Java HTTP server with no web framework in its request path. It accepts posted readings, buffers them, and on a fixed window boundary reduces each berth-and-sensor group to a compact summary carrying the count, minimum, maximum, average, and latest value, then evaluates that summary against the terminal's threshold rules before anything is sent onward. The buffer itself is lock-free: readings are appended under a monotonically increasing sequence number into a single concurrent ordered map, and draining a window snapshots the current sequence boundary and removes exactly the entries recorded before it, so ingestion and draining never contend for the same structure and the "latest" value in a window is unambiguously the last reading in arrival order. Four threshold rules are evaluated, each held as pure data (the sensor type, which aggregate field to read, a comparison operator, a limit, and the alert to raise) and interpreted through two small lookup tables, one mapping a field name to its extractor and one mapping an operator to its comparison, rather than a rule that carries its own embedded logic. A crane-load average above its limit raises a crane-overload alert, a wind-speed average above its limit raises a high-wind crane-halt alert, a berth-occupancy average above its limit raises a congestion warning, and a reefer-temperature average above its limit raises a cold-chain breach. The fifth sensor type, container stack height, deliberately carries no rule: it is one of the five required signals and is shown on the dashboard, but it never alerts. If the outbound send of a finished window fails, the gateway logs the failure and moves on to the next window rather than retrying or buffering locally, since the readings have already been drained and the next window arrives seconds later; an occasional dropped window is an accepted trade-off at this data volume against the complexity of a retry or spill-to-disk layer.

B. Backend: Queue, Function, and Store

The fog gateway never calls the ingestion function directly. Every window's summaries are placed on an Amazon SQS queue instead, the classic decoupling that message-oriented middleware provides between a producer and a consumer whose rates should vary independently without either blocking the other [6]. A direct synchronous call would tie the gateway's throughput to whatever concurrency the function has available at that instant, and a burst that exceeded it would fail with nowhere for the data to wait; the queue absorbs that burst as backlog instead. Publishing is batched: an entire flush cycle's summaries are collected and sent in one batched call, split only where the queue interface caps a batch at ten entries, which keeps the number of publish calls proportional to the number of distinct groups that closed in a window rather than to how many readings arrived. Batching in this way holds the per-message overhead down as message volume grows, and it is the difference this project's own load test was built to exercise.

An event-driven function consumes the queue: it is invoked with a batch of messages, transforms each summary into a stored record, and writes it to DynamoDB. The store is keyed so that the partition key is the sensor type and the sort key joins the window-end time with the berth identifier. Leading the sort key with the window-end time means the store's own key ordering is already chronological, so the dashboard's dominant query, the most recent windows for a sensor type, is a single backward partition read rather than a table scan;

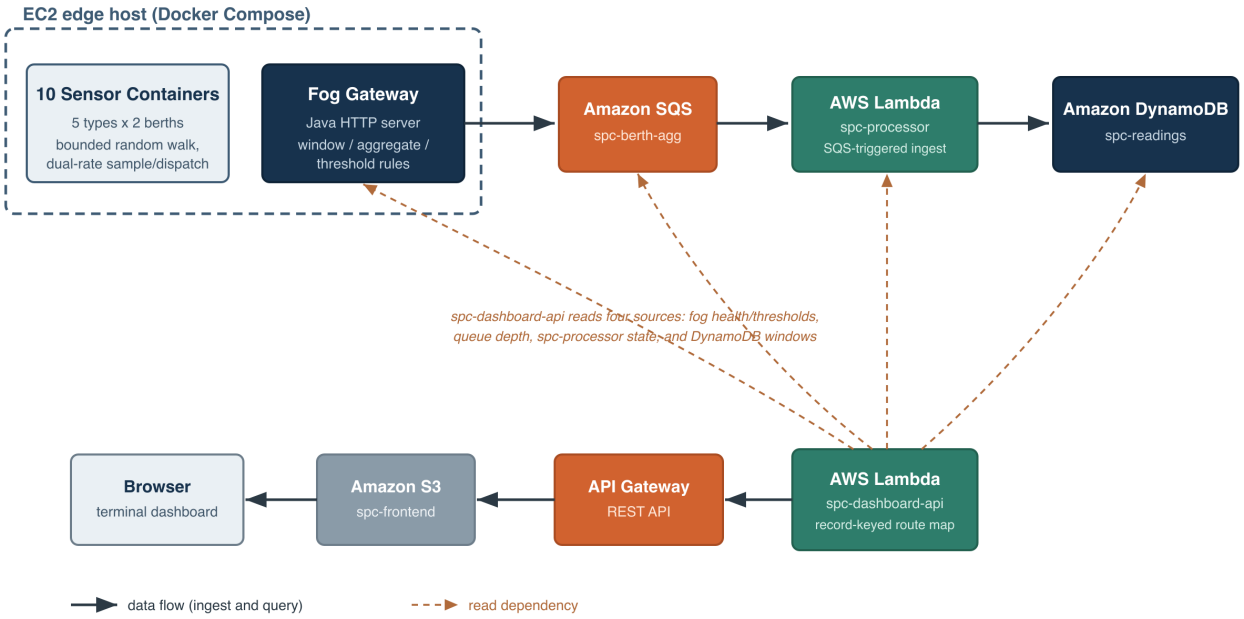

Fig. 1. Deployment topology. Ten sensor containers and the fog gateway share one EC2 edge host. Solid arrows are the data flow: the ingest path runs left to right (fog to SQS to the processor function to DynamoDB), and the query path runs right to left (the dashboard function through API Gateway and S3 to the browser). Dashed edges are the four read dependencies the dashboard function assembles per request: the fog gateway's health and threshold endpoints, the queue's depth, the processor function's deployment state, and the stored windows in DynamoDB.

appending the berth identifier is deliberate, because two berths closing a window for the same sensor type in the same cycle would otherwise collide on an identical key and the second write would silently overwrite the first, keeping writes spread across the key space so that no single partition absorbs a disproportionate share of the traffic. DynamoDB was chosen for exactly the access pattern its key-value lineage was designed around, predictable key-based reads and writes at scale without index tuning as the table grows [7]. Both functions run on demand rather than continuously: the ingestion function fires once per delivered batch and the dashboard function once per request, and neither is billed while idle, which suits a workload that only produces a message when a sensor group actually reported during a window. The known cost of that elasticity is cold-start latency on an idle function [8], but the ingestion path is asynchronous and queue-triggered rather than user-facing, so a cold start delays only when a reading is durably stored, never a page load, and the dashboard's own polling keeps its function warm.

C. Dashboard API and Frontend

The operator view is served by a second function, deployed independently of the ingestion function so the read path and the write path scale and fail separately. It answers a small read-only interface: a per-berth grouping endpoint that returns, for each sensor type, the latest window reported by each berth together with a computed status line; a readings endpoint that returns a recent series for one sensor type to draw the cross-berth trend; a health endpoint; and a backend-statistics endpoint. Its request dispatch is deliberately simple and distinctive: rather than a nested lookup, a chain of conditionals, or a framework router, each request resolves through a single lookup into one flat routing table keyed by a small immutable value of the HTTP method and path together, so a route is found by one map lookup with no scanning. Every response the function returns, including its not-found and error paths, carries an unconditional wildcard cross-origin header, because the frontend is served from an S3 origin while the interface answers from a separate API Gateway origin and a browser would otherwise block the cross-origin reply without a visible network error. The health endpoint reasons about pipeline freshness by reading four independent things directly rather than trusting one cached flag: whether the fog gateway answers, whether the queue is reachable, whether the ingestion function reports itself deployed, and how old the freshest stored window is across every sensor type. That last check is the one that actually detects a stalled pipeline, since a queue and a function can both look healthy while no new data has landed for an hour, and only a timestamp comparison against the store catches it.

The static frontend renders each berth as a card of five metric rows drawn against native meter elements, a headline

operational-status strip reducing each berth to its current alert state, an alert banner naming every active breach, a cross-berth crane-load trend chart drawn with a locally vendored charting library rather than a content-delivery network, and a four-indicator pipeline health strip. A fixed browser-side timer re-fetches the per-berth snapshot, health, backend statistics, and trend series together on a short interval and repaints from whatever those responses hold, so the view stays current without a manual reload or any push channel. The frontend learns which interface origin to call from a single inert element in the page whose text is substituted with the real API Gateway origin at upload time and read once by the page script, a mechanism chosen so the same static files can be uploaded against any deployment's endpoint without a rebuild.

D. Critical Analysis of Alternatives

Several designs were weighed and set aside, each for a stated reason rather than by default. Amazon Kinesis Data Streams was considered in place of SQS: it would keep an ordered, replayable record of every window per shard, which a standard queue does not guarantee, but ordering is unnecessary here because the dashboard reads the latest window per sensor type from the store directly, not a stream of intermediate states, so Kinesis would have added shard-management overhead with no matching need to spend it on. A relational database was considered in place of DynamoDB and set aside for this workload: the access pattern is a narrow, known set of key-based reads and appends, not the ad-hoc joins a relational engine is built for, and choosing the managed key-value store also avoids provisioning, patching, or paying for an always-on instance, keeping the backend serverless end to end. A single function handling both ingestion and dashboard queries was considered in place of two: it was rejected because the two have very different triggers and load profiles, and coupling them would let a slow dashboard query compete for the same concurrency budget as message processing, the exact interference that splitting them avoids. Finally, converting the sensor and fog tier itself to a scheduled-function model, so the whole system and not only the backend would be serverless, was considered and explicitly deferred rather than dismissed: a continuous sensor loop and a stateful windowed buffer map awkwardly onto a stateless, time-limited invocation model without reworking the buffering, and the deployment scope here is the backend layer, so this is recorded as future work rather than attempted partially.

E. Security Posture

The edge host exposes only a single inbound port carrying the fog gateway's health and threshold endpoints; there is no remote-login key pair and no inbound path for interactive access, and administration goes through the cloud provider's systems-management channel, which removes the most common remote-access surface rather than merely narrowing it. What that single port serves is read-only, non-sensitive status, never data access or a control-plane operation. Both functions run under the laboratory account's single shared role because the account's policies block creating new, more narrowly scoped roles: this is a constraint of the environment, not a design preference, and a production account would give the ingestion function only the queue-receive and item-write permissions it needs and the dashboard function only read-oriented permissions. The dashboard interface is intentionally unauthenticated because it exposes only aggregated, non-personal terminal telemetry, well below the threshold where access control would be meaningful at this scale. Each cloud-facing component builds a static credential pair only when an explicit local-emulator endpoint is configured; absent that endpoint, as on the real edge host and inside both real functions, the SDK's default credential chain resolves to the instance profile or the function's execution role, so a fake credential pair never shadows the real one in production.

III. Implementation

A. Software Stack

The entire pipeline is written in Java 17 across four independently built modules: the sensors, the fog gateway, the ingestion processor, and the dashboard. Every network-facing component is built on the standard library alone, its built-in HTTP server for both the fog gateway and the local dashboard profile and its built-in HTTP client for outbound calls, with no third-party web framework anywhere in the request path. The AWS SDK for Java v2 supplies the queue, store, and function clients, and Jackson supplies JSON handling. The dashboard's trend chart uses a charting library vendored into the static assets rather than loaded from a content-delivery network, so the page renders identically whether or not the browser can reach a third-party host. The dashboard module additionally produces a second, cloud-function entry point alongside its local development server, so the same repository, pipeline-check, and threshold-proxy logic serves both a laptop and a real API Gateway integration without a parallel implementation.

B. Automated Test Suite

All 95 tests across the four modules pass, run with JUnit against hand-written fakes of the AWS SDK client interfaces, with no mocking library and no test touching a real AWS endpoint or a running emulator. Table I breaks the suite down by module. Coverage is weighted toward the components with the most decision logic: the fog module carries the bulk of it, exercising the lock-free buffer under concurrent ingest, the window arithmetic, the threshold rules at and around their limits, the batched publisher's chunking behaviour, and the router's ability to distinguish a genuinely absent route from a known path under the wrong method. Two of the fog tests bind the real HTTP server to an ephemeral port and drive it with actual requests over a socket rather than calling a handler in process, which is the only way to prove that a body that is not valid JSON at all, as opposed to a well-formed body missing a field, is rejected by whichever layer parses it first. The dashboard tests exercise every route through the cloud-function entry point with request-shaped events and injected fake clients, asserting on the returned status, headers, and body

TABLE I
AUTOMATED TEST SUITE BY MODULE (95 TESTS TOTAL)

Module	Tests	Coverage focus
sensors	6	bounded random walk, dual-rate sample and dispatch, payload shape
fog	48	lock-free buffer under contention, window arithmetic, threshold rules at boundaries, batched publishing, HTTP ingest over a real socket
backend/processor	8	message-to-item transform, sort-key construction, batch handling
backend/dashboard	33	record-keyed route dispatch, per-berth grouping, pagination fix, health and backend-statistics

directly, and they cover the pagination fix described below with a multi-page scan whose per-page counts must sum correctly rather than stopping at the first page.

C. Continuous Integration

A GitHub Actions workflow runs on every push and pull request as two dependent jobs rather than one. The first installs a Java 17 toolchain and runs the test goal in each of the four modules in turn, so a failure in any one module's own suite fails the build before the second job starts. The second job only runs once the first has passed: it brings up the full stack with Docker Compose against a local AWS emulator and runs an end-to-end verification script against the running containers, confirming that a reading actually reaches the store through the whole pipeline rather than only that individual functions return correct values in isolation, then tears the stack down regardless of outcome so no containers linger between runs.

D. AWS Deployment and Defects Corrected

Every resource the system runs on was provisioned by a single reusable infrastructure-as-code apply, twenty-four resources created in one operation with no manual command-line step in between: the store, the queue, both functions and the event-source mapping that connects the queue to the ingestion function, the full API Gateway chain, the edge host with its security group and fixed public address, and the two S3 buckets with the frontend bucket's public-access configuration and object upload. Table II lists the live resources. Provisioning the whole cloud tier as one declared artifact, rather than a sequence of console or command-line actions, means the resource topology is versioned and reproducible and the dashboard function's fog-endpoint configuration is wired automatically to the address the apply itself allocated, with no value copied by hand between steps.

A code audit carried out before deployment, rather than after, examined the codebase for the failure modes that a local emulator's small fixtures do not surface, and corrected two. The first was an item-count routine behind the backend-statistics endpoint that issued a single count-only scan and returned that scan's count directly, correct only while the table's scanned data stays under the roughly one-megabyte limit a single scan is subject to; beyond that the store signals more to read through a continuation cursor, and a caller

TABLE II
LIVE AWS RESOURCES (US-EAST-1)

Resource	Identifier	Role
DynamoDB table	spc-readings	stores closed-window summaries
SQS queue	spc-berth-agg	decouples fog gateway from the processor
Lambda	spc-processor	queue-triggered ingestion into the store
Lambda	spc-dashboard-api	record-keyed route dispatch behind API Gateway
API Gateway REST API	93xrytbtkb	public entry point for the dashboard interface
EC2 instance	i-038501eeaf331757a	fog gateway + 10 sensor containers
Elastic IP	54.158.198.173	fixed address for the edge host
S3 buckets	spc-frontend-659211701832 / spc-deploy-659211701832	static dashboard / deploy staging

that never follows it silently undercounts. The fix keeps re-issuing the scan from the previous response's cursor and summing the per-page counts until no cursor remains, so the reported total stays correct regardless of table size. The second was unbatched publishing: the flush cycle originally issued one send per closed group, correct but unnecessarily costly whenever more than one group closes in the same window, which is the normal case across two berths and five sensor types; the fix collects a whole cycle's summaries into batched sends chunked at the ten-entry limit and issues them once per cycle. No credential-handling defect was present, because every cloud-facing component already gated its static emulator credentials on the local-emulator endpoint and fell through to the default credential chain on the real account.

E. Live Deployment Verification

Verification took place against the live endpoint, not only against the local suite. Within about a minute of the edge host completing its container bootstrap, the health endpoint reported all four fields true, with the freshest window under two seconds old, meaning a reading generated on a container had already traversed the sensor, the fog gateway, the queue, the ingestion function, and the store, and was being read back. The backend-statistics endpoint showed the stored item count climbing across successive polls rather than a static number, and the per-berth endpoint returned real per-metric data for both berths. All four static frontend assets returned successfully from the S3 bucket, the wildcard cross-origin header was present on the interface responses, and the frontend's inert origin element had been correctly substituted with the real API Gateway origin at upload time rather than left as its placeholder, confirming the configuration mechanism took effect on the deployed files. Fig. 2 shows the deployed dashboard rendering both berths' telemetry with a crane-overload alert firing on berth-a. The capture was taken after the

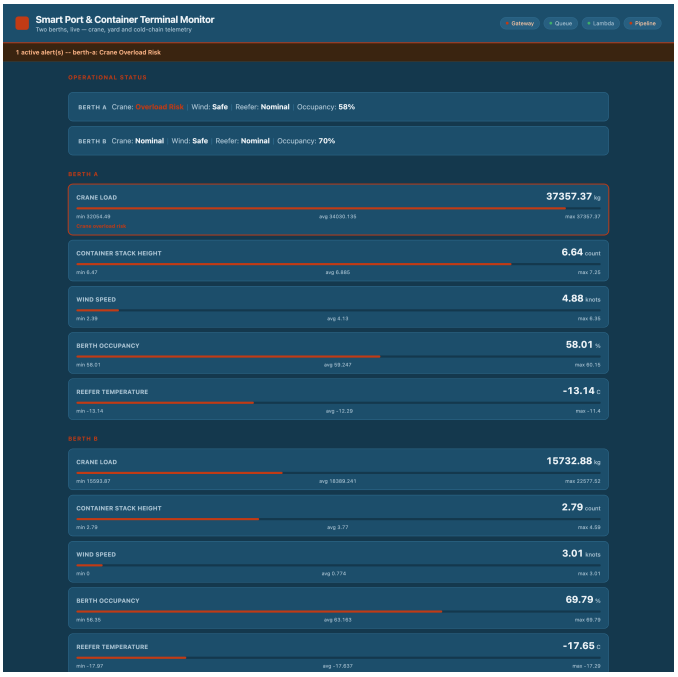

Fig. 2. The deployed dashboard rendering live two-berth telemetry through API Gateway: the operational-status strip, both berths' five metric rows with window minimum, average, and maximum, and a firing crane-overload alert on berth-a. Queue and function indicators and all stored data continue to be served from the serverless tier after the edge host was paused between sessions.

edge host had been paused between sessions, so the gateway and pipeline-freshness indicators reflect that paused edge host while the queue and function indicators, the stored per-berth windows, and the cross-berth history all continue to be served from the serverless tier; this is a direct illustration of the design property discussed in Section IV, that the dashboard and its interface do not depend on the edge host being up.

The implementation, spanning the sensor, fog, processor, and dashboard modules and the infrastructure-as-code that provisions the live account, is available at [GitHub repository URL].

IV. Conclusion

This project set out to move container-terminal awareness from periodic reporting toward continuous, automated observation, and to do so with a distributed design that places computation where it belongs: windowing and rule evaluation at a fog tier beside the equipment, durable storage and elastic serving in the cloud. All four objectives were met. The sensor and fog layer generates five sensor types per berth at independently configurable rates and reduces them to windowed summaries with real threshold evaluation rather than forwarding raw samples. The backend scales through queue decoupling, event-driven functions billed only per request, and an on-demand store, and each of those choices was weighed against a named alternative rather than adopted by default. The dashboard turns the stored windows into a per-berth operational verdict with a firing alert. And the whole pipeline was deployed to a real public-cloud account and verified there directly, not left as a description of one.

Two lessons stand out. The first is that a passing test suite and a correct deployment are different claims: the pagination undercount and the unbatched publishing both passed every local test cleanly, because the fixtures were too small for either symptom to appear, and both were caught only by reading the code against production-scale behaviour rather than by any failing test. The second is architectural: keeping the dashboard and its interface fully serverless, independent of the edge host, meant the operator view kept serving stored telemetry even when the edge host was paused, a property visible directly in the live capture and one that a design resting the dashboard on the same host would not have.

The design also carries real limitations worth stating rather than glossing over. The sensor and fog tier runs on a single edge host, which is a single point of failure for that tier even though everything downstream of it is redundant managed infrastructure; the deferred move to a scheduled-function fog model would close that gap. The laboratory account is session-based, so the evidence here reflects a single continuous session rather than sustained multi-day operation, and a load test has been exercised against the emulator but not yet pointed at the live queue and functions long enough to establish this account's concurrency ceiling. Both functions share one broadly scoped role because the account cannot mint narrower ones, so the least-privilege split described in Section II is specified but not deployed. Beyond closing those gaps, natural extensions are a compound rule that reasons over more than one metric at once, so that a berth simultaneously borderline on crane load and on wind is escalated rather than shown as two independent low-severity states, and per-sensor-type window durations so a fast-moving signal such as wind can be checked more often than a slow one such as reefer temperature without forcing every metric onto the same cadence.

References

[1] R. Stahlbock and S. Voß, "Operations Research at Container Terminals: A Literature Update," *OR Spectrum*, vol. 30, no. 1, pp. 1–52, 2008.

[2] Y. Yang, M. Zhong, H. Yao, F. Yu, X. Fu, and O. Postolache, "Internet of Things for Smart Ports: Technologies and Challenges," *IEEE Instrumentation & Measurement Magazine*, vol. 21, no. 1, pp. 34–43, 2018.

[3] A. V. Dastjerdi and R. Buyya, "Fog Computing: Helping the Internet of Things Realize Its Potential," *Computer*, vol. 49, no. 8, pp. 112–116, 2016.

[4] W. Shi and S. Dustdar, "The Promise of Edge Computing," *Computer*, vol. 49, no. 5, pp. 78–81, 2016.

[5] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50–58, 2010.

[6] E. Curry, "Message-Oriented Middleware," in *Middleware for Communications*, Q. H. Mahmoud, Ed. Chichester, UK: John Wiley & Sons, 2004, pp. 1–28.

[7] R. Cattell, "Scalable SQL and NoSQL Data Stores," *ACM SIGMOD Record*, vol. 39, no. 4, pp. 12–27, 2011.

[8] E. Jonas, J. Schleier-Smith, V. Sreekanti, C.-C. Tsai, A. Khandelwal, Q. Pu, V. Shankar, J. Carreira, K. Krauth, N. Yadwadkar, J. E. Gonzalez, R. A. Popa, I. Stoica, and D. A. Patterson, "Cloud Programming Simplified: A Berkeley View on Serverless Computing," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2019-3, 2019.